A Web-Based Solution for Federated Learning with LLM-Based Automation

Chamith Mawela, Chaouki Ben Issaid, and Mehdi Bennis

arXiv:2408.13010v1 [cs.LG] 23 Aug 2024

***Abstract*—Federated Learning (FL) offers a promising approach for collaborative machine learning across distributed devices. However, its adoption is hindered by the complexity of building reliable communication architectures and the need for expertise in both machine learning and network programming. This paper presents a comprehensive solution that simplifies the orchestration of FL tasks while integrating intent-based automation. We develop a user-friendly web application supporting the federated averaging (FedAvg) algorithm, enabling users to configure parameters through an intuitive interface. The backend solution efficiently manages communication between the parameter server and edge nodes. We also implement model compression and scheduling algorithms to optimize FL performance. Furthermore, we explore intent-based automation in FL using a fine-tuned Language Model (LLM) trained on a tailored dataset, allowing users to conduct FL tasks using high-level prompts. We observe that the LLM-based automated solution achieves comparable test accuracy to the standard web-based solution while reducing transferred bytes by up to 64% and CPU time by up to 46% for FL tasks. Also, we leverage the neural architecture search (NAS) and hyperparameter optimization (HPO) using LLM to improve the performance. We observe that by using this approach test accuracy can be improved by 10-20% for the carried out FL tasks.**

***Index Terms*—Federated learning, distributed computing, web sockets communication, large language models (LLMs), intent-based automation.**

I. Introduction

The recent advancements in machine learning (ML) have led to a surge in organizations adopting these technologies in their business solutions, driven by the abundance of user-generated data. However, the conventional centralized approach to model training poses a threat to user privacy, as raw client data must be shared with an external party. Federated learning (FL) [1] has emerged as a novel paradigm that enables decentralized model training without sharing raw data. FL addresses this issue by enabling collaborative learning where only model weights are communicated among edge nodes and a parameter server (PS), minimizing the risk of private client data leakage. Since its inception by Google [1], FL has been adopted across various domains, such as global COVID diagnosis [2], vehicular networks [3], and wireless communications [4]–[6].

Despite its potential, the adoption of FL is hindered by the complexity of building reliable communication architectures and the need for ML and network programming expertise. In fact, researchers and stakeholders participating in FL implementations must rely on programming expertise and develop seamless communication architectures between edge nodes and the PS. The FL community primarily relies on frameworks such as Flower [7], PySyft [8], and TensorFlow Federated [9], which require both programming and FL domain expertise. Flower and PySyft follow a similar design architecture where the PS and the client services are programmed separately and deployed into respective devices. Flower uses the remote procedure call (gRPC) protocol for communication between clients and PS while PySyft uses WebSockets. TensorFlow Federated on the other hand is developed only for standalone simulation of FL setting on a single device.

In frameworks such as Flower and PySyft, the model architecture and hyperparameters for the FL task are typically hardcoded within the PS or client programs. This setup presents a significant challenge during iterative experimentation, as modifications to the model or its parameters require altering the code on either the PS or client devices. Consequently, only individuals with access to these devices can conduct iterative experiments on the FL process by adjusting the parameters. This motivates us to develop a web-based solution where the clients and PS serve as Application programmable interface (API) endpoints. This eliminates the requirement for direct access to any device to change parameters since it can be done using the API endpoint. Through the provided solution we eliminate the need for programming to carry out FL tasks since the solution will be provided as a web application. Another major drawback of the frameworks mentioned above is the lack of inherent support for model compression schemes. When the size of the model used for the FL process becomes bigger, the possibility of communication bottlenecks occurring could become higher in limited bandwidth communication settings. Hence to alleviate the potential bottlenecks in the FL communication process, our proposed solution will provide support for model compression schemes such as quantization [10], [11] and sparsification [12]. In Table I, we summarize the main difference between our framework and the existing solutions.

More recently, due to the success of Large Language Models (LLMs) and their ability to fine-tune downstream tasks, a wide use of LLMs for automating various tasks can be observed. LLM-integrated tasks span from implementing co-pilots to fully automated solutions. Most of the solutions follow the same structure where an intent or instruction is given by a third party to the LLM where the intent holds necessary information for the LLM to proceed further to produce the intended output. Mostly the instruction is generated by a human or in some cases by the program itself. For instance, LLMs were used to solve math problems [13], automate real-world

C. Mawela, C. Ben Issaid, and M. Bennis are with the Centre of Wireless Communications, University of Oulu, 90014 Oulu, Finland. Email: {chamith.mawelamudiyanselage, chaouki.benissaid, mehdi.bennis}@oulu.fi

TABLE I: Comparison of our proposed framework with existing solutions.

Feature	Our Solution	Flower	PySyft	TensorFlow Federated
Web-based solution (No programming required)	✓	×	×	×
Built-in model compression	✓	×	×	×
Built-in scheduling algorithms	✓	×	×	×
Distributed implementation	✓	✓	✓	×
FL without direct device access	✓	×	×	×
Communication protocol	WebSockets	gRPC	WebSockets	N/A
Intent-based automation of FL using LLM	✓	×	×	×
Automated NAS and HPO for FL	✓	×	×	×

API consumption [14], generate Verilog and Golang code [15], [16], and automate IT infrastructure management using few-shot learning [17] or repair Ansible scripts for edge cloud infrastructure management [18]. To the best of our knowledge, there is no existing research in the literature that addresses the automation of FL through user prompts. This study aims to fill this gap by exploring the potential of leveraging user prompts to automate the federated learning process, making it accessible for users with less FL domain expertise.

In FL tasks, the performance heavily depends on the architecture of the global model and the hyperparameters employed during training. In traditional ML contexts, neural architecture search (NAS) [19] and hyperparameter optimization (HPO) [20] are commonly utilized to identify the optimal model architecture and hyperparameters tailored to a specific dataset. Hence, in our work, we investigate utilizing NAS and HPO to improve the performance of the LLM-based automated solution. Usually, NAS and HPO involve creating a search space for model architectures and hyperparameters, respectively, and carrying out an exhaustive search on the created search space to find the optimal configuration. Due to the dynamic nature of our automated solution and the capability of LLM in producing model architectures, we utilize LLM to create the search space for NAS.

The main challenge in implementing FL tasks using multiple hardware entities as edge nodes and a PS is the need to build a reliable communication architecture from scratch. Our goal is to bridge the gap between conceptualizing and implementing an FL solution. We will address the limitation posed by other frameworks, by making the FL process accessible to parties without direct access to the devices running PS and clients. We also propose a solution to automate the FL process for user prompts, which we believe would make FL tasks more accessible to users with less domain knowledge. Accordingly, we introduce a web-based solution with an efficient communication architecture developed using WebSockets, to orchestrate FL tasks easily. Furthermore, we integrate an intent-based platform into the developed solution, that uses a fine-tuned LLM to carry out federated learning operations based on user prompts. Our contributions are as follows

- We develop a standard web application supporting the FedAvg algorithm [1] for performing FL tasks. A user-friendly web interface is developed so users can easily select the required parameters for the FL tasks. Furthermore, an efficient communication architecture based on WebSockets is developed as the back-end solution to handle communication between the PS and the edge nodes.
- We explore the implementation of secondary functionalities in FL, such as model compression to optimize the transmission of model weights and scheduling algorithms to manage client participation in the FL task.
- We investigate intent-based automation of the developed solution, where federated learning tasks are carried out according to a provided high-level prompt. To achieve this, we fine-tune an open-source LLM on a tailored dataset and integrate the fine-tuned LLM into the developed solution to automate the FL process based on user prompts.
- To further enhance the performance of the automated solution, we implement NAS and HPO. Specifically, we investigate the use of LLM to generate the search space for NAS and develop a selective halving-based approach for the HPO process to find the best model architecture and hyperparameters required for a specific FL task.

The rest of the paper is organized as follows. FL and related secondary functionalities are detailed in Section II. Section III gives a brief introduction to LLM and their fine-tuning process. The system design and the methodology of the proposed solution are discussed in Section IV. An evaluation of the experimental results for the provided solution is discussed in Section V. Finally, the paper concludes with final remarks in Section VI.

II. Federated Learning

FL is a paradigm of ML where models are trained collaboratively using multiple edge nodes (clients), and a PS. Unlike traditional centralized ML training, data is not

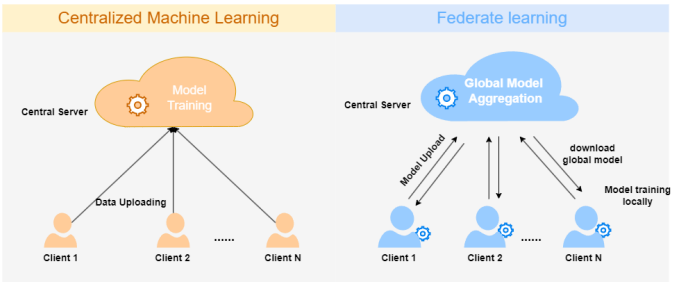


Fig. 1: Centralized Machine Learning vs Federated Learning.

communicated to a central server. Instead, model optimization occurs locally on the clients, preserving data privacy, as shown in Figure 1. The objective of FL is to minimize the weighted average of the local objective functions of each client as follows

$$\min_{w \in \mathbb{R}^d} f(w) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(w), \tag{1}$$

where $F_k(w)$ is the local objective function for the client k. It is assumed here that the total dataset is partitioned among K number of clients where $n_k = |\mathcal{P}_k|$ and $\mathcal{P}_k$ is the set of data points assigned for the kth client. With these objectives in mind, Federated Averaging (FedAvg) has been proposed in [1]. Considered one of the most widely used and influential algorithms in the realm of FL, FedAvg has inspired numerous extensions and improvements. In the remainder of this paper, we restrict ourselves to using FedAvg in our implementation.

One of the major challenges in FL is scalability. As the model size and the number of clients grow, encountering a communication bottleneck in FL training may become inevitable. To alleviate the communication bottleneck related to the model size, model compression schemes have been adopted in the FL literature. Next, we review some common techniques for reducing the size of the models.

A. Quantization

Quantization is a technique for making models compact by reducing the precision of tensors. Usually, the 32-bit precision of *float32* is reduced to a lower precision, e.g., an 8-bit precision of *int8*. A mapping between the floating point x and its quantization x_q is defined as

$$\mathcal{Q} : x \in [\alpha, \beta] \rightarrow x_q \in [\alpha_q, \beta_q]. \tag{2}$$

The quantization function $\mathcal{Q}(\cdot)$ [10] is formulated using *affine mapping* between floating point value and its quantized value, and using the *clip* function as follows

$$\mathcal{Q}(x, S, Z) = clip(round(\frac{x}{S} + Z), \alpha_q, \beta_q), \tag{3}$$

where the clipping function is defined as

$$clip(x, \alpha, \beta) = \begin{cases} \alpha & \text{if } x < \alpha \\ x & \text{if } \alpha \leq x \leq \beta \\ \beta & \text{if } x > \beta. \end{cases} \tag{4}$$

Here, S is an arbitrary positive real number, and Z maps the 0 in floating value to the corresponding value in quantized data type. S and Z represent the scale and zero point quantization parameters, respectively. The recovery of the floating point value through de-quantization can be done using the following equation

$$f_d(x_q, S, Z) = S(x_q - Z). \tag{5}$$

When applying quantization in the FL setting, locally trained models are quantized and communicated to the PS, where de-quantization is done to recover the original model weights before doing the model aggregation.

B. Sparsification

Sparsification [12] is an alternative approach to model compression, which involves selectively retaining only a subset of the gradients through a sparse selection process. The most widely used sparsification methods in FL are Top_k and $Rand_k$ where given a parameter $1 \leq k \leq d$ for a gradient vector $x \in \mathbb{R}^d$, only a k subset of gradients selected will be communicated to the PS.

- $\mathbf{Top_k}$: The top k percent of gradients with the highest absolute values are selected while other values are set to zero. $Top_k : \mathbb{R}^d \rightarrow \mathbb{R}^d$ is formulated as below.

$$[Top_k(x)]_j := \begin{cases} [x]_{\alpha(j)}, & \text{if } j \leq k \\ 0, & \text{otherwise} \end{cases} \tag{6}$$

  Here, α is a permutation of d such that $|[x]_{\alpha(j)}| \geq |[x]_{\alpha(j+1)}|$ for $j \in [1, d-1]$.
- $\mathbf{Rand_k}$: The k percent of gradients are selected based on a random permutation of the list of gradients. $Rand_k : \mathbb{R}^d \times \Omega_k \rightarrow \mathbb{R}^d$ is formulated as below.

$$[Rand_k(x, \omega)]_j := \begin{cases} [x]_j, & \text{if } j \in \omega \\ 0, & \text{otherwise} \end{cases} \tag{7}$$

  Here, ω is a subset chosen from Ω_k where, $\Omega_k = \binom{[d]}{k}$ represents set of all the k-element subsets of $[d]$.

In the FL context, clients communicate only a sparse selection of k percent of gradients and their corresponding values to the PS, thereby improving communication efficiency.

III. Large Language Models

LLMs have revolutionized the AI community. They can understand and generate human-like text across various domains. Models like GPT [21], [22] have made significant progress in Natural Language Processing (NLP) tasks such as machine translation, question-answering, programming, reasoning, and decision-making. This spectacular performance of LLMs can be attributed to the large number of model parameters trained on a huge text corpus typically extracted from the web.

The success of LLMs has been further accelerated by large corporations open-sourcing their pre-trained language models for use by the general public. *Llama* [23] and *Llama2* [24] which spans models with 7B to 70B parameters released by

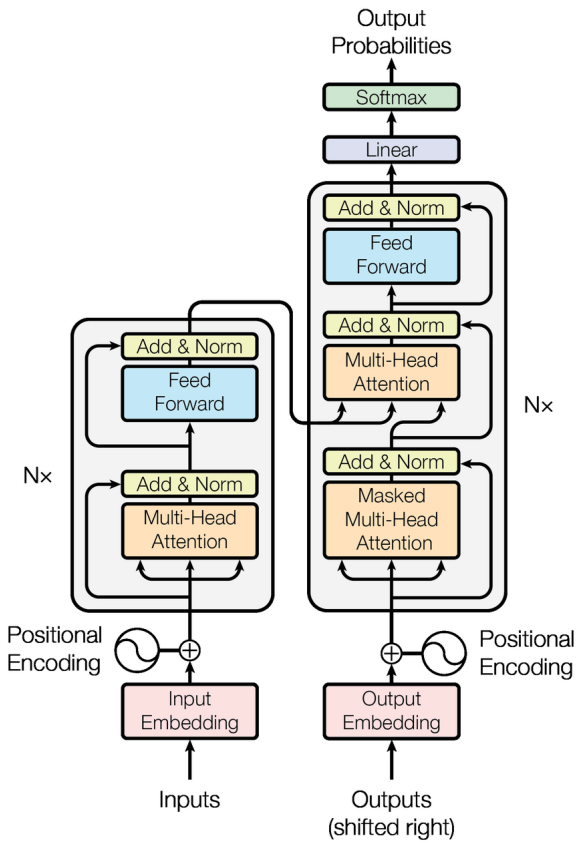


Fig. 2: Transformer architecture: encoder on the left block and decoder on the right block.[1]

Meta AI, and *Mistral 7B* [25], *Mixtral 8x7B* [26] by *Mistral.ai* with parameters ranging from 7B to 13B are some of the widely used open-source LLMs in the community.

A. Transformer Architecture

Every LLM mentioned above occupies some variation of the transformer architecture originally introduced in the paper [27]. As depicted in Figure 2, the transformer architecture is made up of two blocks: (i) an encoder (shown on the left), and (ii) a decoder (shown on the right). The transformer architecture is a powerful tool for NLP tasks. Both the encoder and decoder are composed of multiple layers, allowing for the processing and generation of complex language representations. The input text is first tokenized and converted into embedding vectors. Positional encoding is then added to provide information about the position of each token in the sequence, using the following equations

$$PE_{(pos,2i)} = sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right), \tag{8}$$

$$PE_{(pos,2i+1)} = cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right). \tag{9}$$

Here, pos represents the absolute position of the token in a sequence of tokens, while $i \in [0, d_{model}/2]$ represents the corresponding dimension related to the embedding vector.

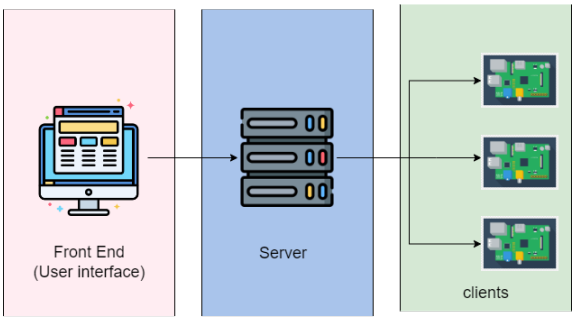


Fig. 3: Overview of the web application FL solution.

The embedded vectors are passed through the multi-head attention layer, which allows the model to focus on different parts of the input simultaneously. The attention mechanism is computed using

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \tag{10}$$

where Q is the query vector, K is the key vector with dimensions d_k, and V is the value vector with dimension d_v acting as the input to the attention function.

The output of the attention layer is then passed through a feed-forward layer and residual connections are used to prevent vanishing gradients. The operation of each sub-layer can be summarized as

$$output = LayerNorm(x + Function(x)), \tag{11}$$

where, x and $output$ represent the input and output of any sub-layer respectively, while $Function$ can be either Multi-Head Attention or Feed-Forward network function.

The decoder block is similar to the encoder but includes a masked multi-head attention layer to ensure that the model only attends to previous tokens when generating new ones. The output of the decoder is passed through a linear layer and softmax function to predict the probabilities of the next token in the sequence. The transformer architecture can be used in various configurations, including encoder-only, decoder-only, and encoder-decoder models, depending on the specific task at hand.

B. Parameter Efficient Fine-tuning of LLMs

Initially, LLMs are trained on large text corpus which possesses general knowledge. Hence, when adopting an LLM for downstream tasks, fine-tuning the model with a tailored dataset specific to the task at hand may be necessary. Fine-tuning dataset usually comes in the form $\{(x_i, y_i)\}_{i=1}^{n}$ where x_i is the input to the model and y_i is the target output from the model. Both x_i and y_i are in the form of a sequence of words or tokens. However, fine-tuning an LLM fully with all the parameters included is a huge computationally expensive task requiring large Gigabytes (GB) of Virtual random access

[1]The transformer model architecture originally appeared in [27] with granted permission to reproduce figures for scholarly works.

memory (VRAM). Therefore, parameter-efficient fine-tuning (PEFT) methods have been explored in the literature.

LoRA [28] is a widely used method for performing efficient fine-tuning on LLMs allegedly reducing trainable parameters by 10000 folds while lowering the Graphical processing unit (GPU) memory requirement by three times compared to fully fine-tuning a GPT-3 175B parameter model with Adam optimizer. LoRA reduces trainable parameters by decomposing the weight matrices using lower-rank matrices. Consider a pre-trained weight matrix $W \in \mathbb{R}^{d\times k}$. In LoRA, the gradient update is represented as follows

$$\Delta W = BA, \tag{12}$$

where $A \in \mathbb{R}^{r\times k}$ and $B \in \mathbb{R}^{d\times r}$, with rank $r << \min(d, k)$. The forward pass is calculated as

$$h = Wx + \Delta Wx = Wx + BAx. \tag{13}$$

where x is the input embedding or intermediate hidden features. Once the fine-tuning is done, inference for the downstream tasks is done by merging the weights as follows

$$W = W + BA. \tag{14}$$

By this means, adapting LoRA for multiple downstream tasks is possible since it is possible to train different LoRA adapters and plug them into the original model as above.

QLoRA [29] is a variant of LoRA to enhance further the efficiency of fine-tuning an LLM. This is achieved by utilizing a quantized version of the transformer model, double quantization, and using paged optimizers. The transformer model is quantized to 4-bit precision using the 4-bit Normal Float (NF4) data type. Hence, QLoRA reduces the GPU memory requirement for fine-tuning a 65B model to less than 48GB, compared to more than 780GB in full fine-tuning.

Algorithm 1 Modified Federated Averaging Algorithm

Inputs: Number of clients K, learning rate η, client fraction C, total communication rounds T, number of local epochs E, mini-batch size B.
Output: Final global model w_{T+1}.
Initialize global model w_0
for each round $t = 1, 2, ..., T$ **do**
  $m \leftarrow \max(C \cdot K, 1)$
  $S_t \leftarrow$ (set of m clients according to the scheduling mechanism selected)
  for each client $k \in S_t$ **in parallel do**
    $\hat{w}^k_{t+1} \leftarrow$ **ClientUpdate**(k, w_t)
    sends $\hat{w}^k_{t+1}$ to server
  end for
  if compress = True **then**
    $w^k_{t+1} \leftarrow \mathbf{DeCompress}(\hat{w}^k_{t+1})$
  end if
  $m_t \leftarrow \sum_{k\in S_t} n_k$
  $w_{t+1} \leftarrow \sum_{k\in S_t} \frac{n_k}{m_t} w^k_{t+1}$
end for

ClientUpdate(k, w)
$\mathcal{B} \leftarrow$ (split $\mathcal{P}_k$ into batches of size B)
for each local epoch i from 1 to E **do**
  for batch $b \in \mathcal{B}$ **do**
    $w \leftarrow w - \eta\nabla\ell(w; b)$
  end for
end for
if compress = True **then**
  $\hat{w} \leftarrow \mathbf{Compress}(w)$
end if

IV. System Design and Methodology

As discussed in previous sections, our goal is to implement a web application to make FL accessible for everyone without dealing with the intricacies of network programming to implement a communication architecture. To that end, we develop a new web application with an inbuilt communication architecture to handle FL tasks. By doing so, we eliminate the need for programming an FL solution from scratch. Our solution is mainly divided into three phases

- **Phase 1:** We develop a standard web solution to facilitate FL tasks.
- **Phase 2:** We fine-tune and integrate an LLM to automate the developed solution further.
- **Phase 3:** We explore NAS and HPO using LLM to improve the results of the automated solution further.

A. Standard Web Application

Our proposed solution comprises three key elements, as illustrated in Figure 3. The front-end user interface[2] is designed to enable end users to input essential parameters required for executing an FL task. The server (here onwards known as FL server) functions as the central coordinator, responsible for tasks such as aggregating parameters, scheduling clients, and managing database services. Additionally, individual clients run separate code locally for training, with each client capable of operating on various hardware setups, provided they possess sufficient computational resources for local training and a communication interface for interacting with the server.

The front-end web user interface is developed as a React [30] application, where Redux [31] is used for managing the state of the front-end data. We use a web form in the front end to get the necessary information to carry out an FL task. Comprehensive information about the most important fields in the web form to be filled by the end user is explained in Appendix VII-A. The user has to submit the required task information via the web form, after which the data is converted to Javascript object notation (JSON) format and transmitted to the FL server to initiate the FL task. An example of the content of a JSON can be found in Appendix VII-B.

WebSockets serve as the communication protocol for all interactions between the front-end and FL server and between the FL server and the clients. WebSockets is a full duplex

[2]The front-end and back-end codes are publicly available at https://github.com/ICONgroupCWC/FedLFrontEnd and https://github.com/ICONgroupCWC/FedLBE, respectively.

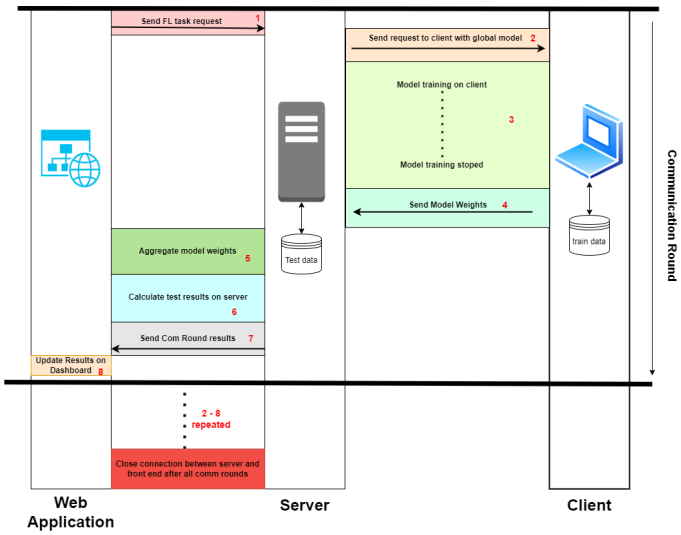


Fig. 4: Communication process between Front-end, server, and clients.

TABLE II: Default values to be used for parameters when unspecified in the prompt.

Key	Value
minibatch	16
algorithm	classification
epoch	5
lr	0.0001 (Adam) 0.001 (SGD) 0.004 (AdaGrad) 0.0004 (RMSProp)
scheduler	full (random if ClientFraction $<$ 1)
clientFraction	1
minibatchtest	32
comRound	10
optimizer	Adam
loss	CrossEntropyLoss
Compress	No

communication protocol built on top of the transmission control protocol (TCP), which facilitates real-time data transfer between two communicating devices. In Appendix VII-C, we explain how the communication flow of WebSockets and the reasons for adopting this choice of communication protocol.

B. Modified FedAvg

To accommodate model compression and client scheduling, we make the following changes to the original FedAvg algorithm

- Model compression schemes are implemented for efficient communication of model weights between the client and the FL server. Users can select a compression scheme to compress the locally trained model before transferring the weights back to the PS. The options for the compression scheme are quantization, Top-k or Rand-k.
- Several scheduling mechanisms are implemented to select clients for each round of communication. Users can choose from the following options
  - **Random:** PS selects clients randomly based on a given client fraction C.
  - **Round Robin:** PS selects clients in a round-robin fashion.
  - **Latency-proportional:** PS selects clients with the lowest latency averaged over k communication rounds.
  - **Full:** PS selects all clients during each communication round.

With these additions in mind, we adapt the implementation of the FedAvg as in Algorithm 1 for our web solution. When the value of the variable $compress$ is set to *False* and the scheduling mechanism is *random*, we recover the original FedAvg algorithm. In Algorithm 1, **Compress** and **DeCompress** are functions implemented on the client and server sides, respectively, for each compression scheme described earlier.

As illustrated in Figure 4, the implementation of the modified FedAvg algorithm leverages Web-Sockets for communication, with eight high-level stages. The process begins with the user submitting an FL request through a web form, establishing a WebSocket connection between the front-end web application and the FL server. Then, the server communicates the request to selected clients based on the client fraction and scheduling mechanism, sending the global model and training parameters. Each client trains the model locally and sends the trained weights back to the server asynchronously. The server aggregates the local models to update the global model, calculates test results, and saves the data in a database. Finally, the current communication round's training and test metrics are sent to the front-end via the existing WebSocket connection, updating the web application dashboard in real-time with user-selected plots. This process is repeated iteratively until all requested communication rounds are completed, maintaining the WebSocket connection between the front-end and FL server throughout the training.

C. LLM Finetuning

The downstream task in our proposed solution is carrying out an FL task based on the user prompt provided. Thus, the LLM needs to be fine-tuned so that the output from the LLM should be compatible with the FL server. In our previous case with the standard web application, the payload in the request from the user contained a JSON, which carries information required for the FL task such as model parameters, optimizer parameters, etc. Hence, it is natural to use the same structure of the JSON as output from the LLM to make the new solution compatible with the previously developed solution. As depicted in Figure 5, to fine-tune the LLM, we create a dataset with <INTENT, TARGET> pairs, where INTENT is the user prompt provided by the user while the TARGET is the

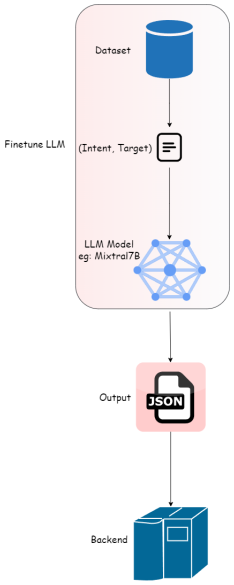


Fig. 5: Fine-tuning process of LLM to be compatible with the FL server.

intended JSON output by the LLM for the provided intent. For example, the user intent can be as follows

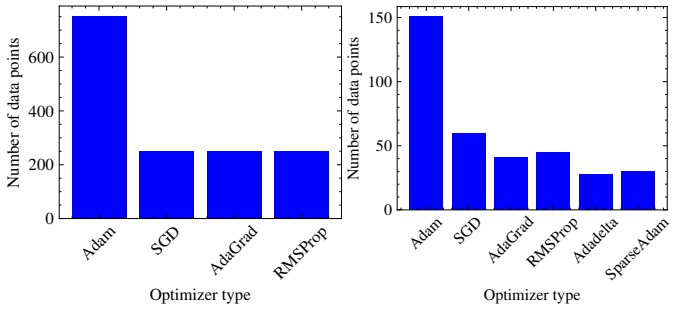
> **Prompt:** *Create a federated learning task with MNIST dataset. Run for a total of 20 communication rounds and 12 local epochs. Use SGD as the optimizer with a learning rate of 0.004. Use a mini-batch size of 32 for training. The client fraction should be 0.7.*

The target JSON should be accustomed according to the provided intent. One caveat here is when the prompt becomes simple, values to be used in the JSON for the unspecified parameters become unclear. To overcome this issue, we use some default values for the parameters whenever the parameter is not specified in the prompt, as specified in Table II. Some of these default values like learning rate are derived experimentally by running for common datasets used in our work such as MNIST.

One other aspect considered when developing the dataset is possible variation in phrasing the user's intent. For example, when the `clientFraction` needs to be specified in the intent, it can be phrased in several ways as follows

> **Prompt:** *Create a federated learning task with MNIST dataset with a client fraction of 0.7.*
> **Prompt:** *Perform a federated learning task with MNIST dataset involving 70% of the total clients.*
> **Prompt:** *Execute a federated learning task with MNIST dataset involving 7/10 of the total clients.*
> **Prompt:** *Start a federated learning task with MNIST dataset by omitting 30% of total clients in each communication round.*

The output JSON of all above prompts should be the same and the dataset is created by including similar variations throughout the dataset for other parameters as well. Apart from that, several other actions are taken to induce variation to the dataset such as paraphrasing, differing complexity of the sentence structure, and differing the order of parameters provided in the intent.



(a) Training dataset (b) Test dataset

Fig. 6: Optimizer variations across the train and test datasets for fine-tuning the LLM.

To get good results when fine-tuning an LLM, structuring the prompt is very important. Apart from the user's intent and the target's response, a system instruction is added to the input prompt. The system instruction can provide contextual guidance to the model, informing it about the task at hand or the desired behavior. This helps the model understand the specific requirements of the fine-tuning task and adjust its predictions accordingly. In our case, the System Prompt is used to communicate the role of the LLM succinctly as shown in Appendix VII-D. A detailed description of each parameter of the output JSON is given by the System Prompt for helping to predict the output accurately. The System prompt is kept static throughout all the data points since we are fine-tuning for a single downstream task and the intended output structure is the same for all the user intents. Finally, each data point is customized using the System Prompt, Intent, and Target before feeding into the fine-tuning pipeline.

D. LLM Integration to the Web Application

Once an open-source LLM model is fine-tuned for our downstream task, the model is integrated into the existing solution. An overview of the extended system design is depicted in Figure 7. We implement a separate server[3] to host the LLM model where a Representational State Transfer (REST) API endpoint is created using Python Flask to consume the LLM service. We convert the fine-tuned LLM model to GGUF format before hosting in the LLM Server, where GGUF is a binary file format used for storing LLM models for inference where loading and saving models are made faster. A Python binding [32] for llama.cpp [33] is used for serving the model inside the provided REST endpoint.

One challenge for the integrated solution is providing the global model architecture for carrying out the FL task. In the standard web application solution, there is the option of submitting a Pytorch model architecture file. However, given that input for the LLM-integrated solution is in natural language, it is not feasible to provide a model architecture

[3]The code for the LLM server is available at https://github.com/ICONgroupCWC/LLMServer

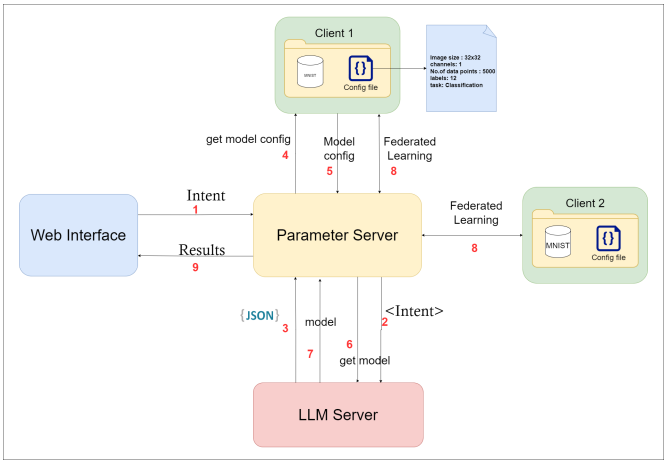


Fig. 7: System design for the LLM integrated FL solution.

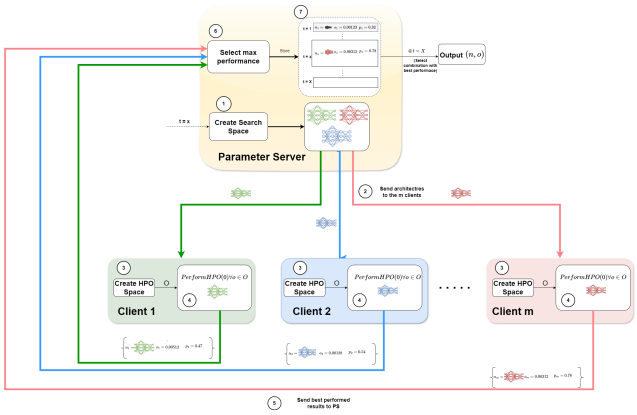


Fig. 8: Federated NAS and HPO process.

as describing the model architecture in natural language is a complex and tedious task. To overcome this, we utilize the reasoning ability of OpenAI ChatGPT to propose a model architecture for a given task. Through experimental results, we verify that given the input and output dimensions of the network and the number of data points, ChatGPT can provide a model architecture accordingly. Since data is a commodity of the clients in the FL process, the responsibility of providing the details of the dataset is given to the clients. A configuration file along with the dataset should be kept in the data folder, which holds information such as the dimension, the number of data points, the task, and the number of labels as depicted in Figure 7. Using this provided configuration file a new prompt is created to query ChatGPT for a model architecture as follows.

> **User_Prompt:** *Create a model architecture for the following task. The task is a Classification task with 12 labels. The input data is a tensor of shape (1,1,32,32) and has 5000 datapoints altogether. Considering the above information, create a neural network architecture that could achieve good accuracy.*

The above user prompt is always kept static, with placeholders for underlined values where the underlined values are overridden using the configuration file provided in the client data folder. Similar to the previous case, a system prompt with task details is used to support this prompt before the request is sent to the ChatGPT API. Accordingly, we implement a separate REST endpoint in the LLM server to query a model architecture for a given task.

As illustrated in Figure 7, the communication process of the integrated solution occurs in nine high-level stages. Initially, the user sends the intent for the FL task to the PS. Next, the PS forwards the intent to the LLM server endpoint to get the configuration of the FL task, where the LLM server predicts the configuration using the fine-tuned LLM and returns the configuration file to the PS as the next step. Then, the PS requests a randomly selected client for the configuration file which contains the dataset's properties. Next, the returned configuration file is forwarded to the LLM server, where the LLM server overrides the aforementioned user prompt and queries the ChatGPT API to provide a model architecture suitable for the task. Once the LLM server receives the model architecture, it is returned to the PS by the LLM server. Finally, the FL process is carried out as usual using the received model architecture as the global model and intermediate results are communicated with the front-end.

E. Neural Architecture Search and Hyperparameter Optimization

As discussed in the previous section, we observe that OpenAI ChatGPT has the ability to suggest a neural network architecture, given the nature of the task and the dataset. However, one caveat in the proposed methodology is achieving good test accuracy is uncertain since the task is carried out using an arbitrary model suggested by the ChatGPT. Hence, in this section, we focus on improving the test accuracy by leveraging ChatGPT to generate the search space for NAS. Also, we perform HPO on the suggested models in the search space to find the best hyperparameters suited for the respective models. Both NAS and HPO are performed in a distributed manner before the FL task starts. The procedure we follow in performing NAS and HPO is provided in Algorithm 2 and illustrated in Figure 8.

According to the provided algorithm, NAS is performed for X number of search rounds as provided in the input. We utilize ChatGPT to initialize the search space at each round, where the **CreateSearchSpace** function will produce m number of models where m is the total number of clients participating in the training at the current search round. Two prompts are used in generating model architectures where at the first round **InitialPrompt** is used while at later rounds **IntermediatePrompt** is used. Also, an **ErrorPrompt** is used to handle errors whenever the models produced by the ChatGPT are invalid. The prompts used are as follows: For all prompts, the underlined values are given as input to the function.

> **InitialPrompt:** *You are an AI that strictly conforms to responses in Python. You are an assistant in providing CNN architectures as a search space for neural architecture search. I need help designing the optimal CNN architecture for a specific dataset. I*

plan to start with a variety of models and change the model architecture based on model performance. As the initial step, please provide 3 CNN architectures with varying width, depth, and layer types. The input data is in the form torch.zeros((1,3,32,32)) and the output is in the form torch.size((1,10)). There are altogether 10000 data points for training. When suggesting models give importance to the above factors to make the model complex accordingly. After you suggest a design, I will test its performance and provide feedback. Based on the results of previous experiments, we can collaborate to iterate and improve the design. Please avoid suggesting the same design again during this iterative process. Your responses should contain valid Python code only, with no additional comments, explanations, or dialogue. Provide the PyTorch models according to the given prompt. Important!! Output the solution as 3 different Python code bases. Start each code base with <Code> and end the code base with </Code> brackets. So there must be 3 brackets. Include PyTorch imports with 'import torch', 'from torch import nn', and 'import torch.nn.functional as F'. Do not include model initialization code.

Intermediate Prompt: *By using provided model model, we achieved an accuracy of 57%. As previously Please recommend 3 new models that outperform prior architectures based on the above-mentioned experiments on neural architecture search. Take a step towards making the model more complex by adding more layers to the neural network and increasing the number of parameters in the model. The output should be structured the same as in the previous case. As in the previous case, the input is in the form torch.zeros((1,3,32,32)), and the output is in the form torch.size((1,10)). Please make sure that all the dimensions are correct in the network and no error will arise*

Error Prompt: *The suggested model (model) gives the following error (error message). Please suggest a new model architecture with similar parameter complexity that conforms with the dimensions of the input and output correctly to avoid the above error. The input is in the form torch.zeros((1,1,32,32)) and output a tensor in the form torch.size((1,10))*

During each search round, the produced m model architectures are sent to the m number of local clients to carry out HPO. For each client, HPO is run for H number of rounds, where at each round, Y hyperparameter configurations are created. In our study, we only focus on learning rate as a hyperparameter as we believe learning rate has the most effect on model performance. Furthermore, focusing on a single hyperparameter significantly reduces the search space. This simplification allows for a more thorough exploration of learning rate values within the given computational constraints. Hence, Y number of learning rates between a provided upper bound of 5×10^{-2} and a lower bound of 10^{-5} will be generated randomly by **CreateHPOSpace** function at each HPO round by each local client.

Algorithm 2 Algorithm for NAS and HPO for automated solution

Inputs: Number of search rounds X, number of HPO rounds H, number of explorer epochs E, total number of clients K, client fraction C, Hyperparameter types T, number of hyperparameters Y, data configurations d, initial batch size B
Output: Final best model n, Best hyperparameters o
for each round $t = 1, 2, ..., X$ **do**
  $m \leftarrow \max(C \cdot K, 1)$
  $N_t \leftarrow$ **CreateSearchSpace**(m, d)
  $S_t \leftarrow$ (set of m clients selected randomly)
  for each client $k \in S_t$ **in parallel do**
    $n_k \in N_t \leftarrow$ (receives model architecture from Server)
    for each round $j = 1, 2, ..., H$ **do**
      $O \leftarrow$ **CreateHPOSpace**(T, Y)
      $o_j, p_j \leftarrow$ **PerformHPO**(n_k, O, E, B)
    end for
    $o_k, p_k \leftarrow$ (where $k = \arg\max p_j$)
    sends (n_k, o_k, p_k) to server
  end for
  $\{(n_k, o_k, p_k) \mid k = 1, 2, 3, \ldots, m\}$ (receives from clients)
  $(n_t, o_t, p_t) \leftarrow$ (where $t = \arg\max_{1 \le k \le m} p_k$)
end for
$(n_f, o_f, p_f) \leftarrow$ (where $f = \arg\max_{1 \le t \le X} p_t$)
$n, o \leftarrow (n_f, o_f)$

PerformHPO(n, O, E, B)
$p_b = 0$ // Stores best performance
$o_b = \{\}$ // Stores best hyperparameter combination
$current_round = 0$
while $O.length > 1$ **do**
  if $current_round \neq 0$ **then**
    $O \leftarrow$ (Select the top performing half of hyperparameters)
    $B \leftarrow min(B \cdot 2, traindata.size)$
  end if
  $\beta \leftarrow$ (select B datapoints randomly from training data)
  for each combination $o \in O$ **do**
    for each epoch $e = 1, 2, 3 \ldots, E$ **do**
      TrainModel(n, o)
    end for
    $p \leftarrow$ **EvaluateModel**(n, o, β)
    if $p > p_b$ **then**
      $p_b \leftarrow p$
      $o_b \leftarrow o$
    end if
  end for
  $current_round += 1$
end while
return (o_b, p_b)

Fig. 9: Experimental setup: three Raspberry PIs, a laptop, and a wireless router.

TABLE III: A summary of the different use cases used in our experiments using MNIST, Fashion-MNIST, CIFAR-10, and SVHN datasets.

Cases	Description
Case 1	An FL task using **MNIST** dataset, with a client fraction = 0.7, local minibatch size = 4, number of local epochs = 1, communication rounds = 50, optimizer = Adam, learning rate = 0.0001
Case 2	An FL task using **CIFAR-10** dataset with a client fraction = 0.7, local minibatch size = 8, number of local epochs = 2, communication rounds = 100, and optimizer = Adam, learning rate = 0.001.
Case 3	An FL task using **Fashion-MNIST** dataset, with a client fraction = 0.8, local minibatch size = 8, number of local epochs = 1, communication rounds = 50, optimizer = Adam, learning rate = 0.0001
Case 4	An FL task using **SVHN** dataset with a client fraction = 0.7, local minibatch size = 8, number of local epochs = 1, communication rounds = 50, and optimizer = Adam, learning rate = 0.0001.

The created HPO space is passed to the **PerformHPO** function where the HPO operation is carried out using a selective halving manner. At each step after the first step, hyperparameter space is reduced by half size where only the best 50% of the hyperparameters per their performance are retained. Meanwhile, the training set size is also increased by two folds to ensure the retained hyperparameter space is trained on more data points while the HPO progresses. The model architecture received by the local client is trained for E number of epochs, for each retained hyperparameter using the batch of data β in the local client.

Once the training is done, the model is evaluated using a validation dataset to find the best-performing hyperparameter. Here, we use the validation accuracy as the performance metric. The best candidate is updated at each round where the **PerformHPO** process is carried out until only one candidate from the HPO space is retained with the highest performance. This procedure is run for H number of HPO operations where the hyperparameter and performance pairs will be recorded. At the end of all the H HPO rounds, the best-performing hyperparameter will be selected from the pool of recorded hyperparameters. Then, the combination of the model, best hyperparameter, and its performance is sent to the server where the server stores these combinations in a list for future comparison. Once all the NAS rounds are completed, the model and hyperparameter combination with the best performance is selected as (n, o) to be used as the global model and learning rate for the specific FL process.

V. Experimental Results

A. LLM Finetuning Results

For the fine-tuning part, we start by creating a new dataset with 1504 data points with <Intent, Target> pairs for fine-tuning the LLM[4]. The dataset is created while considering a wider range of possible variations that could occur in the user intent. As illustrated in Figure 6, several variations of optimizer choices are included in the dataset. As observed in the plots, the number of instances containing default values is higher. This is due to the higher number of data points not specifying the relevant parameter in the intent. For example, it is natural for a user to input only the `clientFraction`, but not the `scheduler`, where in such cases the default value is used.

This dataset is used to fine-tune a Mixtral-7B [25] model. The fine-tuning process is carried out for two epochs through the dataset using the QLoRA method[5]. Hence, the model is loaded in 4-bit precision for fine-tuning, where at the end of the training, the LoRA module is merged with the base model by upscaling the trained LoRA to 8-bit precision. We use an 8-bit precision variation of adamW [34] optimizer, Adamw_8bit, as the optimizer with a learning rate 2×10^{-4} for fine-tuning. A single A100 GPU provided by Google Colab is used for fine-tuning where Unsloth [35] library is used for efficient fine-tuning. Unsloth is an open-source library that provides Python API for faster fine-tuning of LLMs.

To evaluate the performance of the fine-tuned Language Model, we create a new test dataset comprising 355 instances similar to the fine-tuned dataset. The dataset is designed to span from simpler prompts with minimal specified parameters to complex prompts with many parameters included, ensuring a comprehensive evaluation spectrum. Additionally, for categorical parameters such as optimizers, new values were added to test the generalization capability of the LLM as evident from Figure 6, where Adadelta and SparseAdam were not used as optimizers in the intents in the fine-tuning stage. Similarly, values for the *dataset* key were also given unseen values with random dataset names to evaluate the generalizability.

[4]The fine-tuned dataset is available at https://huggingface.co/datasets/cmarvolo/auto_fl.

[5]The fine-tuned LoRA adapter is available at https://huggingface.co/cmarvolo/mistral-7b-fed-auto-lora.

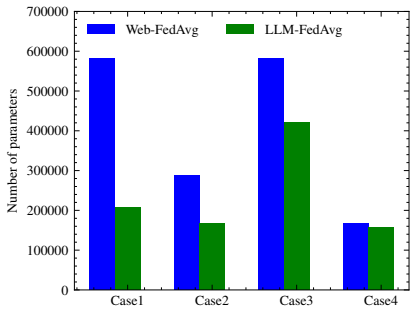


Fig. 10: Number of parameters of the model used for Web-FedAvg and LLM-FedAvg for each case.

TABLE IV: Performance of Web-FedAvg and LLM-FedAvg for the different cases.

Cases	Method	Test Accuracy	Communication Overhead (bytes)	CPU (s)
Case 1	Web-FedAvg	98.69%	1.165×10^2	3.761×10^3
	LLM-FedAvg	98.14%	4.150×10	2.004×10^3
Case 2	Web-FedAvg	69.18%	6.700×10	4.290×10^4
	LLM-FedAvg	72.55%	6.276×10	3.880×10^4
Case 3	Web-FedAvg	89.74%	1.165×10^2	1.736×10^3
	LLM-FedAvg	89.69%	8.450×10	1.649×10^3
Case 4	Web-FedAvg	86.30%	3.350×10	1.643×10^4
	LLM-FedAvg	89.87%	3.150×10	1.583×10^4

The evaluation accuracy of the test dataset holds at 99.4%, where only two data points are incorrectly predicted.

B. Comparative Analysis of Web-FedAvg and LLM-FedAvg Implementations

Several simulations are carried out using both standard and LLM-integrated solutions to evaluate the results. In the remainder of this paper, we denote by *Web-FedAvg* and *LLM-FedAvg* the standard solution and the LLM-integrated solution, respectively. For all the simulations, three Raspberry Pi modules are used as clients and both the PS and the LLM server are run on a single Laptop with an i7-9750H CPU, 24GB RAM, and a NVIDIA Geoforce 1660Ti GPU with 6GB dedicated memory, where all the devices are connected to a single local wireless network. The hardware setup used in the simulations is illustrated in Figure 9.

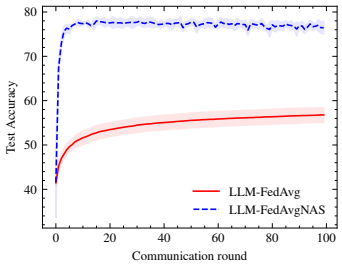
Simulations are run for several cases using MNIST, CIFAR-10, Fashion-MNIST, and SVHN datasets equally distributed among the clients. A summary of scenarios used for simulations is given in Table III. For the cases represented in the table, *LLM-FedAvg* method is carried out by creating a relevant prompt to capture the necessary parameters. The simulation results are tabulated in Table IV for both *Web-FedAvg* and *LLM-FedAvg* methods, where communication overhead and CPU time represent the total number of bytes transferred from the client to the server and the total elapsed time for all the communication rounds, respectively. The number of parameters of the models used by both methods is plotted in Figure 10, where for *Web-FedAvg*, a handcrafted model is used, while for *LLM-FedAvg*, the model is suggested by ChatGPT given the dataset configuration.

From case 1, it is observed that both cases achieve test accuracy over 98% where *Web-FedAvg* achieves slightly better accuracy. However, communication overhead and CPU time are much less for *LLM-FedAvg*. For case 2, *LLM-FedAvg* seems to achieve around 3% better test accuracy than *Web-FedAvg* while maintaining a lesser communication overhead and CPU time. Observing case 3, we can see that both *Web-FedAvg* and *LLM-FedAvg* achieve a similar accuracy of around 89% with *LLM-FedAvg* achieving the results with lesser communication overhead and CPU time. Finally, in case 4, *LLM-FedAvg* manages to achieve around 3% better accuracy over *Web-FedAvg* while having lesser communication overhead and elapsed CPU time. In summary, both *Web-FedAvg* and *LLM-FedAvg* seem to achieve similar accuracy for simulations carried out, while sometimes *LLM-FedAvg* achieve slightly better accuracy. However, for all the cases communication overhead and elapsed CPU time for the FL process is lesser in *LLM-FedAvg* than *Web-FedAvg*. The reason is that chatGPT continuously suggests models with lesser parameters as depicted in Figure 10 for *LLM-FedAvg* for all the cases.

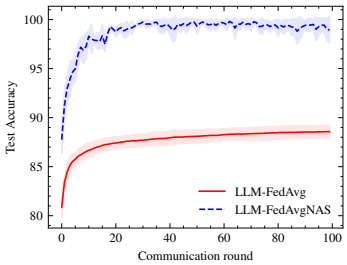
C. Comparative Analysis of LLM-FedAvg and LLM-FedAvgNAS Implementations

Although *LLM-FedAvg* manages to achieve a considerably good test accuracy, we believe the accuracy could be further improved. Hence, we utilize NAS and HPO as described in Algorithm 2, termed as *LLM-FedAvgNAS* here onwards. We only perform NAS and HPO on CIFAR-10, Fashion-MNIST, and SVHN datasets where MNIST is omitted as a higher accuracy over 98% is already achieved. For all the *LLM-FedAvgNAS* search processes, five search rounds (S) with two local HPO rounds (H) are carried out. Only the learning rate is used as the hyperparameter type (T). The total number of epochs (E) is kept as 20 while a client fraction of one is set to use all three clients in the search process. The number of hyperparameters (Z) is set as 20, where for each HPO round, 20 random learning rates are created as HPO search space. The initial batch size for all the simulations is kept at 250. Due to the increased compute and memory demand for the search process, the clients, in this case, are run using a single V100 GPU since Raspberry PI compute is insufficient to carry out the search process.

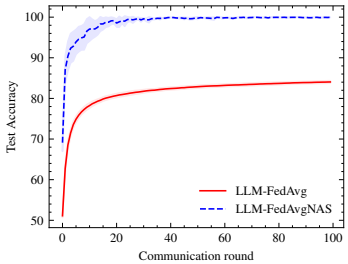
Once the best model and learning rate are obtained using *LLM-FedAvgNAS* search, the FL process is carried out separately using the obtained results for 100 communication rounds and only one local epoch with a client fraction of 0.7 to utilize only two of three total clients. *LLM-FedAvgNAS* search is carried out three times obtaining three global model and learning rate pairs to plot the mean and standard deviation of the test accuracy of FL communication rounds as illustrated in Figure 11. Similarly, *LLM-FedAvg* is carried out three times using the same number of communication rounds and client fraction. The prompt used for this only specifies the dataset, communication rounds, local epochs, and client fraction ensuring that default values specified in



(a) CIFAR-10 dataset



(b) Fashion-MNIST dataset



(c) SVHN dataset

Fig. 11: Results comparison between LLM-FedAvg and LLM-FedAvgNAS for CIFAR-10, Fashion-MNIST and SVHN datasets.

TABLE V: Average times for single search round in *LLM-FedAvgNAS* for different datasets.

Dataset	**CPU(s)**
CIFAR-10	1.7651×10^3
Fashion-MNIST	1.2127×10^3
SVHN	1.7995×10^3

Table II are used for the rest of the parameters. Similar to the previous case, the mean test accuracy and standard deviation for *LLM-FedAvg* simulations are plotted in Figure 11.

Observing the results in Figure 11, we can see that *LLM-FedAvgNAS* surpasses in test accuracy by a large margin *LLM-FedAvg*. For the CIFAR-10 dataset, *LLM-FedAvgNAS* manages to achieve a test accuracy of around 76% while for Fashion-MNIST and SVHN datasets, an optimal accuracy of over 98% is achieved. Hence, we conclude that although the *LLM-FedAvg* method provides decent results, incorporating NAS using LLM and performing HPO, the performance of the provided automated solution can be improved further. Although *LLM-FedAvgNAS* offers better performance, it comes with a significant computational overhead during the offline search process before starting the FL task as illustrated in Table V, which summarizes the average CPU time taken for a single search round for each dataset.

VI. Conclusion

In this work, we introduced a web-based solution for orchestrating FL tasks, designed to be accessible to researchers without extensive programming knowledge. Complex communications for seamless support for FL tasks are implemented using an efficient WebSocket architecture. Moreover, the integration of intent-based automation, achieved through fine-tuning an LLM, further enhances user accessibility by allowing FL processes to be automated via natural language prompts. Experimental results demonstrate that our automated solution achieves comparable or superior performance to standard methods while maintaining the communication overhead up to 64% lower and CPU time up to 46% lower for the considered scenarios. We also integrated NAS and HPO to enhance performance. Notably, the incorporation of NAS and HPO led to significant performance improvements where we observed a 10-20% test accuracy increase over the test cases run, effectively addressing potential limitations of the automated approach, although with an added computational overhead in the search process.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

[2] Y. Xu, L. Ma, F. Yang, Y. Chen, K. Ma, J. Yang, X. Yang, Y. Chen, C. Shu, Z. Fan, J. Gan, X. Zou, R. Huang, C. Zhang, X. Liu, D. Tu, C. Xu, W. Zhang, D. Yang, M.-W. Wang, X. Wang, X. Xie, H. Leng, N. Holalkere, N. J. Halin, I. R. Kamel, J. Wu, X. Peng, X. Wang, J. Shao, P. Mongkolwat, J. Zhang, D. L. Rubin, G. Wang, C. Zheng, Z. Li, X. Bai, and T. Xia, "A collaborative online ai engine for ct-based covid-19 diagnosis," *medRxiv*, 2020. [Online]. Available: https://www.medrxiv.org/content/early/2020/05/19/2020.05.10.20096073

[3] K. Tan, D. Bremner, J. L. Kernec, and M. Imran, "Federated machine learning in vehicular networks: A summary of recent applications," in *2020 International Conference on UK-China Emerging Technologies (UCET)*, 2020, pp. 1–4.

[4] S. Niknam, H. S. Dhillon, and J. H. Reed, "Federated learning for wireless communications: Motivation, opportunities, and challenges," *IEEE Communications Magazine*, vol. 58, no. 6, pp. 46–51, 2020.

[5] M. M. Amiri and D. Gündüz, "Federated learning over wireless fading channels," *IEEE Transactions on Wireless Communications*, vol. 19, no. 5, pp. 3546–3557, 2020.

[6] A. Elgabli, J. Park, C. B. Issaid, and M. Bennis, "Harnessing wireless channels for scalable and privacy-preserving federated learning," *IEEE Transactions on Communications*, vol. 69, no. 8, pp. 5194–5208, 2021.

[7] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão *et al.*, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[8] T. Ryffel, A. Trask, M. Dahl, B. Wagner, J. Mancuso, D. Rueckert, and J. Passerat-Palmbach, "A generic framework for privacy preserving deep learning," 2018.

[9] TensorFlow Developers , "Tensorflow federated," 2019. [Online]. Available: https://www.tensorflow.org/federated

[10] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 2704–2713.

[11] C. B. Issaid, A. Elgabli, J. Park, M. Bennis, and M. Debbah, "Communication efficient decentralized learning over bipartite graphs," *IEEE Transactions on Wireless Communications*, vol. 21, no. 6, pp. 4150–4167, 2021.

[12] N. F. Eghlidi and M. Jaggi, "Sparse communication for training deep networks," *arXiv preprint arXiv:2009.09271*, 2020.

[13] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[14] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian *et al.*, "Toolllm: Facilitating large language models to master 16000+ real-world apis," *arXiv preprint arXiv:2307.16789*, 2023.

[15] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "Verigen: A large language model for verilog code generation," *ACM Trans. Des. Autom. Electron. Syst.*, feb 2024, just Accepted. [Online]. Available: https://doi.org/10.1145/3643681

[16] Q. Gu, "Llm-based code generation method for golang compiler testing," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2023. New York, NY, USA: Association for Computing Machinery, 2023, p. 2201–2203. [Online]. Available: https://doi.org/10.1145/3611643.3617850

[17] J. Lin, K. Dzeparoska, A. Tizghadam, and A. Leon-Garcia, "Appleseed: Intent-based multi-domain infrastructure management via few-shot learning," in *2023 IEEE 9th International Conference on Network Softwarization (NetSoft)*, 2023, pp. 539–544.

[18] S. Kwon, S. Lee, T. Kim, D. Ryu, and J. Baik, "Exploring llm-based automated repairing of ansible script in edge-cloud infrastructures," *Journal of Web Engineering*, vol. 22, no. 06, p. 889–912, Dec. 2023. [Online]. Available: https://journals.riverpublishers.com/index.php/JWE/article/view/24673

[19] P. Ren, Y. Xiao, X. Chang, P.-y. Huang, Z. Li, X. Chen, and X. Wang, "A comprehensive survey of neural architecture search: Challenges and solutions," *ACM Comput. Surv.*, vol. 54, no. 4, may 2021. [Online]. Available: https://doi.org/10.1145/3447582

[20] M. Feurer and F. Hutter, "Hyperparameter optimization," *Automated machine learning: Methods, systems, challenges*, pp. 3–33, 2019.

[21] L. Floridi and M. Chiriatti, "Gpt-3: Its nature, scope, limits, and consequences," *Minds and Machines*, vol. 30, pp. 681 – 694, 2020. [Online]. Available: https://api.semanticscholar.org/CorpusID:228954221

[22] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[23] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[24] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[25] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, "Mistral 7b," *arXiv preprint arXiv:2310.06825*, 2023.

[26] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[28] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.

[29] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[30] React Developers , "React framework," 2024. [Online]. Available: https://react.dev/

[31] Redux Developers , "Redux framework," 2024. [Online]. Available: https://redux.js.org/

[32] A. Betlen, "llama.cpp binding for python," 2024. [Online]. Available: https://github.com/abetlen/llama-cpp-python

[33] G. Gerganov, "llama.cpp library," 2024. [Online]. Available: https://github.com/ggerganov/llama.cpp

[34] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *arXiv preprint arXiv:1711.05101*, 2017.

[35] D. Han, "Unsloth," 2023. [Online]. Available: https://github.com/unslothai/unsloth.git

VII. Appendices

A. Web Form Fields

The input parameters by the end user to the web form and their description are shown in the following table.

TABLE VI: Fields used in the web form and their description.

Field Name	Description
Task Name (*text*)	Unique identifier for the task
Host IP (*text*)	IP address of the PS
Client list (*text*)	List of client IP addresses
Algorithm type (*dropdown*)	Classification Regression
Local minibatch size (*int*)	Minibatch size for local training
Local epochs number (*int*)	Number of local training iterations
Learning rate (*decimal*)	Learning rate used for training
Scheduler Type (*dropdown*)	Random Round robin Latency proportional Full
Client Fraction (*decimal*)	Fraction of clients participating in a single round of communication
Test Minibatch size (*int*)	Minibatch size for testing
Communication rounds number (*int*)	Total number of rounds between the PS and the clients
Model file (*file*)	Python model architecture file
Optimizer (*dropdown*)	Adam SGD ...
Loss function (*dropdown*)	Cross entropy loss MSE loss ...
Compression scheme (*dropdown*)	Quantize Topk Randomk No compression

B. JSON File Content

The LLM should produce a compatible JSON output when a user needs to run an FL task through an intent-based method. The associated JSON file for this task is given as follows

```
{"algo": "Classification",
 "minibatch": "16",
 "epoch": "5",
 "lr": "0.0001",
 "scheduler": "full",
 "clientFraction": "1",
 "minibatchtest": "32",
 "comRounds": "10",
 "optimizer": "Adam",
 "loss": "CrossEntropyLoss",
 "compress": "No",
 "dataset": "MNIST"}
```

C. Communication Flow of WebSockets

Unlike in HTTP protocol, WebSocket connection is long-lived, where the connection is closed only when one party deliberately closes the connection or the connection times out due to idle data flow. Initially, an HTTP request is sent to the server from the client requesting a connection upgrade to WebSockets. Websockets connection is established

upon connection upgrade response from the server where the connection is kept until the connection is closed by either party. The choice of adopting Websockets for our use case is primarily due to,

- **Bi-Directional data transfer:** WebSockets enable bi-directional communication between the Server and the Client. Since in an FL setting, model weights must be communicated between the Server and Client and vice versa, this bi-directional communication enables efficient synchronization of model updates.
- **Real-time communication:** WebSockets facilitate real-time communication by maintaining a persistent connection between the client and the server. Unlike traditional Hypertext Transfer Protocol (HTTP), which requires establishing a new connection for each request, WebSockets leverage an established connection, eliminating the overhead associated with connection setup. This real-time communication is advantageous for FL, where rapid exchange of model updates is crucial for collaborative learning.
- **Long-lived connection:** WebSockets connection is long-lived, meaning the connection will exist until a client deliberately closes the connection or the connection is inactive for a longer period. This allows in our use case where connections between the user interface, FL server, and clients must be kept alive until respective FL tasks are carried out, which could be minutes or hours in duration.

D. System Prompt

The System Prompt is used to explain the LLM's task. It is kept static and supplemented with each intent given to the LLM, both during fine-tuning and inference. The System Prompt used in our work is given as follows

> **System Prompt:** *Your task is to provide a JSON given an instruction by the user for a federated learning task. The output should be in JSON format. Keys for JSON are algo - Classification/Regression, minibatch - size of minibatch (16), epoch- number of epochs(5), lr - learning rate, scheduler - full/random/round_robin/latency_proportional(full), clientFraction- fraction of clients involved in federated learning (1),comRounds- number of communication rounds in federated learning(10), optimizer-pytorch optimizer(Adam), loss(CrossEntropyLoss), compress-No/quantize(No), dtype-img(img),dataset-dataset used for training. Default values for each key are given inside the bracket. Separated by / are possible values for the relevant key. Your task is to create a JSON with the above keys and extract possible values from a given human prompt as values for the JSON. Respond only to the JSON.*